Samuel González-Castillo

Mathematical Structures Within Simple Type Theory

Abstract. We present an extension of simple type theory that incorporates types for any kind of mathematical structure (of any order). We further extend this system allowing isomorphic structures to be identified within these types thanks to some syntactical restrictions; for this purpose, we formally define what it means for two structures to be isomorphic. We model both extensions in NFU set theory in order to prove their relative consistency.

Keywords: Simple type theory, NFU set theory, Mathematical structures, Higher-order logic.

Mathematics Subject Classification: 03B16, 03B38, 03E75.

1. Introduction

The introduction of the *ramified theory of types* by Bertrand Russell led to the subsequent formulation of several variants of the theory. A notable example is the *simple theory of types* (also referred to as "simple type theory") introduced by Alonzo Church in 1940 [5]. Simple type theory is a versatile and expressive framework, the many virtues of which have already been discussed in the literature [8]. Moreover, the study of its extensions has been an active area of research.

Moving to a different dimension, an essential notion in the development of modern mathematics has been that of *mathematical structure*. Examples of mathematical structures include rings, groups, vector spaces, topological spaces and posets. Informally, mathematical structures are often defined as "sets with some additional structure on them subject to some properties".

Presented by **Francesco Paoli**; *Received* April 14, 2023

Studia Logica (2025) 113: 1513–1542
https://doi.org/10.1007/s11225-024-10133-1
© The Author(s) 2024

In this paper, we seek to formulate an extension of simple type theory—with set-theoretical semantics—in which it be possible to work with mathematical structures and to identify them under isomorphism. We will accomplish this in a two-stage process that we will now succinctly outline.

Firstly, we will extend the language of simple type theory and introduce the "language of structural type theory"; this extension will revolve around "structural types"—a new kind of types that will be included in the language. We will define some semantics for this language, and we will introduce a formal system that will extend simple type theory and be sound with respect to those semantics. We shall call the resulting theory "weak structural type theory", and we will refer to these particular semantics of the language as its "weak semantics". In the framework of weak structural type theory, it will be possible to work with mathematical structures, but not to identify them under isomorphism.

Lastly, we will tweak the weak semantics of the language of structural type theory into what we will call the "strong semantics" of the language. Under them, it will be possible to identify isomorphic structures within structural types. Hence we will further extend the formal system of weak structural type theory in order to reflect this fact and take advantage of it, thus defining what we will call "(strong) structural type theory". We will of course show that this extension preserves consistency and is sound with respect to the strong semantics of the language.

In terms of related work, we should mention that the goals that we pursue can be achieved with other systems. The type-theoretical treatment of mathematical structures can be accomplished with System F [9,14], and the additional goal of identifying isomorphic structures can be achieved within homotopy type theory [15] through its powerful univalence axiom. Structural type theory focuses on the goals that we have outlined and aims to be simpler than these systems; it also benefits from being equipped with some natural set-theoretical semantics.

2. Preliminaries

In this section, we sketch the language, semantics and deductive apparatus of simple type theory, and we briefly present the formal system of NFU set theory (which will be used later in the interpretation of the language of structural type theory). Detailed expositions of these theories can be found in the references.

Regarding notation, given any formal construction F and any pair of symbols a and b, we will write $F[a\|b]$ to denote the result of non-recursively replacing every occurrence of a in F by b. It should be highlighted that this replacement operator will also affect subscripts, as in $x_A[A\|B] := x_B$. Moreover, we may also write $F[a_1\|b_1, \ldots, a_n\|b_n]$ to denote the *greedy*, simultaneous, non-recursive replacement of the occurrences in F of each construction a_i by b_i (for $i = 1, \ldots, n$). By "greedy", we mean that, if a_i is part of a_j and there is an occurrence of a_j in F, then it is a_j that gets replaced by b_j in $F[a_1\|b_1, \ldots, a_n\|b_n]$ (and, consequently, any instances of a_i which were part of a_j are ignored). For example,

$$(x_A, y_A)[x_A\|t_A, A\|B] := (t_A, y_B).$$

Here, A is part of x_A, so x_A gets replaced by t_A and the replacement $A\|B$ has no effect on x_A.

2.1. Simple Type Theory

Simple type theory, as we will consider it, was first introduced by Church [5] and further developed in two articles by Henkin [10] and Andrews [2]. Exhaustive and self-contained expositions of simple type theory can be found in a book by Andrews [3, Ch. 5] or in a more recent survey by Farmer [8].

A language of simple type theory consists of a set $\mathcal{E}$ of *expressions* and a set $\mathcal{T}$ of *types* together with an assignment $t : \mathcal{E} \longrightarrow \mathcal{T}$. We write $a : A$ to denote that $t(a) = A$ (and, implicitly, that $a \in \mathcal{E}$ and $A \in \mathcal{T}$) and we read it as "a is (an expression) of type A".

The set of types of such a language is defined from a countable collection $\mathcal{A}$ of *atomic types* as follows:

1. There is a type $*$, which we refer to as the *type of truth values*.
2. Every atomic type is a type ($\mathcal{A} \subseteq \mathcal{T}$).
3. If A and B are types, so is the *type of functions* $A \longrightarrow B$.

Then, the set of expressions is defined using an infinite collection of *variable symbols* $\mathcal{X}$ with a distinguished element $\perp \in \mathcal{X}$ and, for each type $A \in \mathcal{T}$, a finite collection of constants $\mathcal{C}_A$. From this point on, the set of expressions, together with the assignment function t, is constructed according to the following inductive rules:

1. *Constants*: For every $A \in \mathcal{T}$ and every $c \in \mathcal{C}_A$, we have $c : A$.
2. *Variables*: Given any variable symbol $x \in \mathcal{X}$ and any type A, the *variable* x_A is of type A. For reasons that will become clear later, given any type

A, we will refer to the variable $\perp_A$ as the *canonical undefined variable* of type A.

3. *Lambda abstraction*: If A and B are types, $x_A : A$ is a variable and $b : B$, then $(\lambda x_A .\ b)$ is of type $A \longrightarrow B$.
4. *Function application*: If A and B are types, $f : A \longrightarrow B$ and $a : A$, then $f(a)$ is an expression of type B.
5. *Equality*: If a_1, a_2 are expressions of the same type, then $(a_1 = a_2) : *$.
6. *Choice operator*: If A is a type, $x_A : A$ is a variable and $\varphi : *$, then $(\epsilon x_A .\ \varphi)$ is an expression[1] of type A.

The formulas of the language are the expressions of type $*$.

We should emphasise that, if A and B are different types and $x \in \mathcal{X}$ is a variable symbol, then x_A and x_B are different expressions with different types (and, in particular, they are different variables). Variable symbols are not variables: they are mere symbols which, only when subscripted with a type, become variables. However, by a convenient abuse of notation, when a variable has multiple instances in an expression, we will sometimes refer to it by its variable symbol (thus omitting the subscript stating its type) after its first occurrence, provided that there are no instances of other variables in the expression using the same variable symbol. For example, if x is a variable symbol and A is a type, we could write $(\lambda x_A .\ x)$ to represent $(\lambda x_A .\ x_A)$. It is also worth remarking that, if we make an informal declaration of the form "let $v : X$ be a variable", it should be understood that v is representing a construction of the form x_X for some variable symbol x.

Also in connection with variables, we will say that an instance of a variable x_A is *bound* whenever it is contained in a (sub)expression of the form $(\lambda x_A .\ b)$ or $(\epsilon x_A .\ \varphi)$; else, it will be said to be *free*. When working in the language of simple type theory (or any extension thereof), given any expression α, any variable $z : Z$ and any expression $\zeta : Z$, we will use $\alpha[z \| \zeta]_{\mathsf{f}}$ to denote the expression resulting from non-recursively replacing every free instance of z in α by ζ.

Given a variable x_A and an expression a of the same type, we shall say that a is *free for* x_A in an expression β if no instance of a free variable in a would become bound in $\beta[x_A \| a]_{\mathsf{f}}$.

---

[1] As the axioms of simple type theory and the semantics of the language will soon reveal, the construction $(\epsilon x_A .\ \varphi)$ is—informally speaking—meant to represent an element of A that satisfies φ when x_A takes its value. If such an element does not exist, then $(\epsilon x_A .\ \varphi)$ is meant to represent $\perp_A$, i.e., the canonical undefined variable of type A

From the basic constructions that we have incorporated into our language, one can define two expressions $\mathsf{T}, \mathsf{F} : *$ representing the two truth values in addition to the usual propositional connectives $(\neg, \wedge, \vee, \rightarrow, \leftrightarrow)$ and logical quantifiers $(\forall, \exists, \exists!)$; the details are discussed in Farmer's article [8, § 4]. We will write quantifications in the same way as lambda abstractions and applications of the choice operator: e.g., we will write $(\forall x_A.\ b)$ for a variable x_A and for $b : *$. Given any expression b, any variable x_A and a symbol $\square$ out of $\lambda, \epsilon, \forall, \exists, \exists!$, the *scope* of the expression $(\square x_A.\ b)$ is b.

Remark 1. For convenience, we may occasionally drop parentheses provided that doing so leads to no ambiguity. For this, we will follow two parsing rules. First, when parsing an expression, we will make the scope of any subexpression of the form $(\square x_A.\ b)$ as big as possible. Second, we will follow the usual order of precedence of operators $(=, \neg, \wedge, \vee, \rightarrow, \leftrightarrow)$. In this way,

$$y = a \rightarrow \forall x_C.\ (b = d) \wedge \exists u.\ \varphi \vee \psi$$

should be parsed as

$$(y = a) \rightarrow \Big(\forall x_C.\ \Big((b = d) \wedge (\exists u.\ (\varphi \vee \psi))\Big)\Big).$$

With this in mind, we may now introduce the axiom schemata of simple type theory.

(T1) $\forall f_{*\longrightarrow *}.\ (f(\mathsf{T}) \wedge f(\mathsf{F})) \leftrightarrow (\forall x_*.\ f(x))$.

(T2) Given any type A, $\forall x_A, y_A.\ (x = y) \rightarrow (\forall p_{A\longrightarrow *}.\ p(x) \leftrightarrow p(y))$.

(T3) For any types A and B, $\forall f_{A\longrightarrow B}, g_{A\longrightarrow B}.\ (f = g) \leftrightarrow (\forall x_A.\ f(x) = g(x))$.

(T4) Given any types A and B, any variable $x : A$ and any expression $b : B$ in which an expression $a : A$ is free for the variable x, $(\lambda x.\ b)(a) = b[x\|a]_{\mathsf{f}}$.

(T5) Given any type A, any variable $x : A$ and any $\alpha : *$ such that $(\epsilon x.\ \alpha)$ is free for x in α, $(\exists x.\ \alpha) \rightarrow \alpha[x\|(\epsilon x.\ \alpha)]_{\mathsf{f}}$ and $\neg(\exists x.\ \alpha) \rightarrow (\epsilon x.\ \alpha) = \bot_A$.

In addition to these axioms, the formal system of simple type theory incorporates one inference rule. Given any two expressions a and b of the same type and a formula $\varphi : *$, from $a = b$ and φ, we may deduce any formula φ' obtained upon replacing one instance of a in φ by b as long as such an instance is not a variable immediately preceded by λ or ϵ. Symbolically, $\{a = b, \varphi\} \vdash \varphi'$.

Regarding semantics, simple type theory can be interpreted naturally in set theory. Every type A in a language of simple type theory can be interpreted as a non-empty set $|A|$ and every expression $a : A$ can be interpreted as an element $|a| \in |A|$. In order to define an interpretation, we only need to interpret every atomic type and its constants. From then on, we can extend our interpretation to all the types of our language by the following inductive rules:

1. The type of truth values needs to be interpreted as a set of two elements, $|*| := \{0, 1\}$.
2. Given any two types A and B, $|A \longrightarrow B|$ is the set of functions from $|A|$ to $|B|$.

Moreover, if we consider a variable assignment function α taking every variable $x_A : A$ to an element $\alpha(x_A) \in |A|$, we can evaluate all the expressions in our language according to the following rules:

1. Given any variable x_A, its evaluation $|x_A|_\alpha$ is $\alpha(x_A)$.
2. Given any constant c, $|c|_\alpha := |c|$.
3. The evaluation of any expression of the form $(\lambda x_A \mathbf{.}\ b)$, with $x_A : A$ and $b : B$, will be a function from $|A|$ to $|B|$ taking every $a \in |A|$ to the evaluation $|b|_{\alpha'}$, where α' is equal to α but for $\alpha'(x_A) := a$.
4. Given any $f : A \longrightarrow B$ and $a : A$, we have $|f(a)|_\alpha := |f|_\alpha(|a|_\alpha)$.
5. Given two expressions a_1 and a_2 of the same type, $|a_1 = a_2|_\alpha := 1$ if $|a_1|_\alpha = |a_2|_\alpha$ and $|a_1 = a_2|_\alpha := 0$ otherwise.
6. Given any expression of the form $(\epsilon x_A \mathbf{.}\ \varphi)$, let C be the collection of elements $a \in |A|$ such that $|\varphi|_{\alpha'} = 1$ where α' is equal to α but for $\alpha'(x_A) := a$. If $C \neq \varnothing$, then the evaluation of $(\epsilon x_A \mathbf{.}\ \varphi)$ will be the element of C chosen by a global choice function; naturally, the same global choice function must be used in all the evaluations of this kind of expressions. Else, if $C = \varnothing$, the evaluation will be $|\bot_A|_\alpha = \alpha(\bot_A)$. This—which is captured by (T5)—is why we refer to $\bot_A$ as the canonical undefined variable of type A.

A formula $\varphi : *$ is true under a given interpretation if, for any variable assignment function α, we have $|\varphi|_\alpha = 1$. This concludes our discussion of the semantics of simple type theory.

Regarding extensions, we could include finite Cartesian products into simple type theory [7]. For this, we could incorporate constants for inclusion

and projection functions, in addition to defining their behaviour axiomatically. Other more sophisticated constructions such as subtypes could also be incorporated into simple type theory [6].

For obvious reasons, given any two types A and B, we refer to the expressions of type $A \longrightarrow B$ as functions with domain A and codomain B.

Let x and y be some variable symbols. Given any sequence of functions $f_1, \ldots, f_n$ with $n \geq 2$ and $f_i : A_i \longrightarrow A_{i+1}$, we define their *composition* (as an object in simple type theory) as

$$f_n \circ \cdots \circ f_1 := \left(\lambda x_{A_1}.\ f_n(\cdots f_1(x)\cdots)\right) : A_1 \longrightarrow A_{n+1}.$$

Given any $f : A \longrightarrow B$, we can construct a formula

$$\text{bijective}[f] := (\forall y_B.\ \exists! x_A.\ f(x) = y)\,,$$

which is true if and only if, for any variable assignment function α, $|f|_\alpha$ is bijective; furthermore, we can trivially define analogous formulas for injectivity and surjectivity. Lastly, using the choice operator, we can define the inverse of any bijective $f : A \longrightarrow B$ as

$$f^{-1} := (\lambda y_B.\ (\epsilon x_A.\ f(x) = y))\,.$$

2.2. NFU Set Theory

The set theory of New Foundations with Urelements (NFU) was introduced by Jensen [12] as an extension of the New Foundations previously formulated by W. V. O. Quine. A very detailed and thorough exposition of NFU set theory can be found in a book by Holmes [11].

We will build a formal system for NFU set theory in a first-order language $\mathcal{Q}$ incorporating equality, a binary atomic relation $\in$, a unary predicate S which we shall read as "is a set", and a unary function symbol ι (informally, it will take any element x to the singleton $\{x\}$). In addition, given any pair of terms x and y, we will introduce the ordered pair (x, y) as a term of $\mathcal{Q}$.

In regard to the introduction of ordered pairs into NFU, an article by Adlešić and Čačić [1] provides an alternative approach.

Remark 2. When parsing first-order formulas, we will follow two rules analogous to those of Remark 1: maximise the length of the scope of every quantifier and then apply the usual order of precedence of operators.

Let φ be a formula in $\mathcal{Q}$ and let $t_1, \ldots, t_n$ be all the terms appearing in φ. We say that φ is a *stratified* formula if we can define a *relative height*[2] function $\sigma : \{t_1, \ldots, t_n\} \longrightarrow \mathbb{Z}$ such that, for every $i, j, k, l = 1, \ldots, n$,

- if there exists an occurrence of $t_i \in t_j$ in φ, then $\sigma(t_i) = \sigma(t_j) - 1$,
- if there exists any occurrence of $t_i = t_j$ in φ, then $\sigma(t_i) = \sigma(t_j)$,
- if a term t_i is (t_j, t_k), then $\sigma(t_i) = \sigma(t_j) = \sigma(t_k)$,
- and, if a term t_i is $\iota(t_j)$, then $\sigma(t_i) = \sigma(t_j) + 1$.

The formal system of simple type theory is constructed over first-order logic with equality and in the language $\mathcal{Q}$. Its axioms are the following:

(N1) If two sets A and B have the same elements, they are equal:

$$\forall A, B.\ (S(A) \wedge S(B) \wedge (\forall x.\ x \in A \leftrightarrow x \in B)) \rightarrow A = B.$$

(N2) If x is not a set, it has no elements:

$$\forall x.\ \neg S(x) \rightarrow \forall y.\ y \notin x.$$

(N3) Ordered pairs are not sets, and $(a, b) = (x, y)$ if and only if $a = x$ and $b = y$:

$$\forall a, b.\ \neg S((a, b)) \wedge \forall x, y.\ (a, b) = (x, y) \leftrightarrow (a = b \wedge x = y).$$

(N4) For every x, the term $\iota(x)$ represents[3] the singleton containing x:

$$\forall x.\ \forall y.\ y \in \iota(x) \leftrightarrow y = x.$$

(N5) Given any stratified formula φ, the set $\{x \mid \varphi\}$ of all the elements x verifying φ exists:

$$\varphi \text{ stratified:} \quad \exists A.\ S(A) \wedge \forall x.\ x \in A \leftrightarrow \varphi.$$

(N6) For every set P containing pairwise disjoint sets, there exists a set C containing a single element from every set in P.

---

[2] What we call "relative height" is often referred to as "relative type". We have tweaked this terminology in order to prevent any possible confusion between notions of set theory and type theory. This apparent coincidence in terminology, however, is not such, for the history of simple type theory and NFU set theory can be traced back to a common origin.

[3] For convenience, we have introduced the ι function symbol and defined it axiomatically. We should nevertheless mention that any atomic formula φ with an instance of a term $\iota(t)$ could be equivalently stated, for a variable $\hat{x}$ that is not free in φ, as $\forall \hat{x}.\ (\forall y.\ (y \in \hat{x}) \leftrightarrow (y = t)) \rightarrow \varphi'$ where φ' is identical to φ but for the replacement of the aforementioned occurrence of $\iota(t)$ by $\hat{x}$.

These axioms allow us to construct, for example, a universe set $V = \{x \mid x = x\}$ or the empty set $\varnothing = \{x \mid x \neq x\}$. In addition, the existence of unions, intersections and Cartesian products can be easily shown. Moreover, we can construct functions between two sets as a subset of their Cartesian product and also consider the set of all functions between two sets. Regarding choice, we are able to define a global choice function that returns, for every non-empty set, a singleton with one of its elements.

Given any set A in NFU, we can construct its power set $\mathcal{P}(A)$. Likewise, we can also consider the set $\mathcal{P}_1(A)$ of all the singletons of elements in A, and, inductively, we may define the set $\mathcal{P}_{n+1}(A)$ of all the singletons with elements in $\mathcal{P}_n(A)$.

Using stratified comprehension, we know that the sets

$$\mathrm{NSet} := \{x \mid \exists y.\ y \in x\}, \qquad \mathrm{NSet}^2 := \mathrm{NSet} \cap \mathcal{P}(\mathrm{NSet}).$$

exist in NFU. They are, respectively, the set of all non-empty sets and the set of all non-empty sets whose elements are all non-empty sets.

Setting $\iota^1 := \iota$, we can inductively define, for every natural number n, $\iota^{n+1}(x) := \iota(\iota^n(x))$ for every term x. In this way, $y = \iota^2(x)$ can be informally interpreted as $y = \{\{x\}\}$. Clearly, for the formula $y = \iota^n(x)$ to be stratified with a height function σ, we need to have $\sigma(y) = \sigma(x) + n$. In addition to this, we will also write $y = \pi^n(x)$ to mean $\iota^n(y) = x$; hence, informally, $\pi^1(\{\{x\}\}) = \{x\}$. Obviously, for every x, there can be at most one y such that $y = \pi^n(x)$.

In our formal system for NFU, we have only introduced ordered pairs of elements, but we may define n-tuples for $n \geq 2$ as $(a_1, \ldots, a_n) := (a_1, (a_2, \ldots (a_{n-1}, a_n) \cdots))$.

3. Weak Structural Type Theory

We are pursuing two main goals. Firstly, we seek to incorporate new types into simple type theory that be home to all the mathematical structures of any given kind. Secondly, we aim for equality within those types to work in such a way that two structures be equal if and only if there exists an isomorphism between them. We will work on the first of these objectives in this section, and we will do so by extending the language of simple type theory with a new notion: that of *structural types*. We will equip the resulting language with a semantic interpretation and define a deductive system upon it, which will give rise to what we will call *weak structural type theory*. This theory will not yet be able to identify isomorphic structures—we will take care

of that in the following section with the introduction of *(strong) structural type theory*. Weak and strong structural type theory will share the same language and rules of inference, but they will have different semantics—which we will refer to as *weak* and *strong* semantics, respectively—and different axioms.

The first question that we need to address is how to define what a mathematical structure is. A possible answer to this question is that a structure is just some kind of mathematical object defined on one or more domains. For instance:

- A magma is nothing more than a law of composition $f : A \longrightarrow (A \longrightarrow A)$ (equivalently, $A \times A \longrightarrow A$) defined on some domain A.
- If we forget about its axioms, a topological space is a collection of subsets $\tau \in \mathcal{P}(\mathcal{P}(X))$ defined over a domain X. In the language of simple type theory, a topological space over a domain X would be defined by a $\tau : (X \longrightarrow *) \longrightarrow *$ since, given a type X, its "power set" can be identified with the type $X \longrightarrow *$.

With these examples in mind, let us formally define the language of structural type theory.

3.1. Syntax & Deductive Apparatus

Definition 3. A *language of structural type theory* is defined by first considering a language of simple type theory containing an infinite collection of atomic types that we will refer to as *type variables*; subject to the condition that no type containing type variables may have constants. We should remark that the language may have more atomic types in addition to these type variables.

From this foundation, the language is extended through the following inductive rules:

1. *Structural types.* If $X_1, \ldots, X_n$ are type variables and F is a type (possibly with instances of these variables), then

$$\Sigma := \coprod_{X_1,\ldots,X_n} F \tag{$*$}$$

is a type which we will label as a *structural type*; we say that the appearances of the type variables X_i in Σ are *bound*, while any other instances of type variables (in any type) will be said to be *free*. The type F must have no instances of $\coprod$; in other words, F cannot contain structural types itself. Furthermore, no type with occurrences of $\coprod$ may have constants.

2. *Structures.* Let Σ be a structural type as in $(*)$. Given any types[4] $A_1, \ldots, A_n$ and any expression $f : F[X_1 \| A_1, \ldots, X_n \| A_n]$, the *structure* $f@\Sigma$ is an expression of type Σ. If x is a free variable in f and there are occurrences of $f@\Sigma$ in any expression b, then the constructions $(\lambda x.\ b)$ and $(\epsilon x.\ b)$ are forbidden in our language; in other words, none of the free variables in $f@\Sigma$ can be bound in a bigger expression.
3. *Structural lambda abstraction.* Let Σ be as in $(*)$ and let $\alpha : M$ be an expression. If $x : F$ is a variable, then the construction $(\lambda x@\Sigma.\ \alpha)$ is an expression of type $\Sigma \longrightarrow M$ if the following conditions are satisfied:
   (a) The expression α does not contain any instances of the choice operator ϵ.
   (b) There are no free occurrences of the type variables X_i in M (the type of α).
   (c) The expression α does not have any free occurrences of any variables, other than x, having free instances of the type variables X_i in their type.
   (d) The expression α does not contain any instances of the symbol @.

Remark 4. Just like any other type, structural types have variables: if x is a variable symbol and Σ denotes a structural type, x_Σ is a variable of type Σ. Moreover, nothing forbids us from using ordinary lambda abstraction with structural types. For example, the function $(\lambda x_\Sigma.\ x_\Sigma)$ would be the identity function on Σ.

Example 5. Let $\Sigma := \coprod_X X$. The expression T, which we introduced in Section 2.1, is of type $*$. Clearly, as $*$ is $X[X\|*]$, we have $\mathsf{T}@\Sigma : \Sigma$. Furthermore, as Σ is $X[X\|\Sigma]$, we also have $\mathsf{T}@\Sigma@\Sigma : \Sigma$.

Using ordinary lambda abstraction, we can construct the expression

$$(\lambda x_\Sigma.\ (\lambda y_\Sigma.\ y_\Sigma)) : \Sigma \longrightarrow (\Sigma \longrightarrow \Sigma),$$

which maps any element in Σ to the identity function on Σ. Moreover, we can also construct

$$(\lambda x_X@\Sigma.\ x_X = x_X)\,.$$

However, we cannot construct $(\lambda x_X@\Sigma.\ x_X@\Sigma)$ as this would breach Condition 3d in Definition 3.

If we also consider the structural types $\Sigma_1 := \coprod_{X,Y} X \longrightarrow Y$ and $\Sigma_2 := \coprod_X X \longrightarrow X$, then, given any $f : A \longrightarrow A$ for some type A, we will have

[4] There are no restrictions on these types. For example, some of them could be structural types, even Σ itself.

$f@\Sigma : \Sigma$, $f@\Sigma_1 : \Sigma_1$ and $f@\Sigma_2 : \Sigma_2$. Nevertheless, if we take an expression $g : A \longrightarrow B$ for a type B different from A, then $g@\Sigma : \Sigma$ and $g@\Sigma_1 : \Sigma_1$, but $g@\Sigma_2$ cannot be constructed.

Lastly, let us highlight that, if $\Sigma' := \coprod_{X_1,\ldots,X_n} F$ is a structural type (for some type variables X_i and for a type F containing no structural types), then, for any $f : F$, we have $f@\Sigma' : \Sigma'$.

The axioms of weak structural type theory are the following:

(T1) $\forall f_{*\longrightarrow *}.\ (f(\mathsf{T}) \wedge f(\mathsf{F})) \leftrightarrow (\forall x_*.\ f(x))$.

(T2) Given any type A, $\forall x_A, y_A.\ (x = y) \rightarrow (\forall p_{A\longrightarrow *}.\ p(x) \leftrightarrow p(y))$.

(T3) For any types A and B, $\forall f_{A\longrightarrow B}, g_{A\longrightarrow B}.\ (f = g) \leftrightarrow (\forall x_A.\ f(x) = g(x))$.

(T4) Given any types A and B, any variable $x : A$ and any expression $b : B$ in which an expression $a : A$ is free for the variable x, $(\lambda x.\ b)(a) = b[x\|a]_{\mathsf{f}}$.

(T5) Given any type A, any variable $x : A$ and any $\alpha : *$ such that $(\epsilon x.\ \alpha)$ is free for x in α, $(\exists x.\ \alpha) \rightarrow \alpha[x\|(\epsilon x.\ \alpha)]_{\mathsf{f}}$ and $\neg(\exists x.\ \alpha) \rightarrow (\epsilon x.\ \alpha) = \bot_A$.

(T6) Given any structural type $\Sigma := \coprod_{X_1,\ldots,X_n} F$, any variable symbol[5] x, any type B, and any expressions $b : B$ and $s : F[X_1\|A_1, \ldots, X_n\|A_n]$ for some types $A_1, \ldots, A_n$, let

$$x^A := x_F[X_1\|A_1, \ldots, X_n\|A_n], \qquad b^A := b[X_1\|A_1, \ldots, X_n\|A_n].$$

If $(\lambda x_F@\Sigma.\ b)$ can be constructed and if s is free for x^A in b^A, we have[6]

$$(\lambda x_F@\Sigma.\ b)(s@\Sigma) = b[x_F\|s, X_1\|A_1, \ldots, X_n\|A_n]_{\mathsf{f}}.$$

In addition, if b has no free occurrences of x_F or x_Σ and no free occurrences of the type variables X_i, then $(\lambda x_F@\Sigma.\ b) = (\lambda x_\Sigma.\ b)$.

The only rule of inference of weak structural type theory is the very same substitution under equality of simple type theory.

As a brief remark, we can mention that the substitution

$$b[x_F\|s, X_1\|A_1, \ldots, x_n\|A_n]_{\mathsf{f}}$$

[5]Recall that a variable of a type A is always of the form x_A for a variable symbol x.

[6]In Section 2.1, we introduced the notation $[-\|-]_{\mathsf{f}}$ to denote a replacement that ignores bound variables. By extension, now that we also have the notions of free and bound type variables, we use the same notation to denote a replacement that ignores both bound variables and bound type variables.

in (T6) always returns a valid construction in the language of structural type theory. This can easily be proven by induction; notice that Conditions 3a–3d in Definition 3 simplify the proof.

3.2. Semantics

We will now define the weak semantics of the language of structural type theory, but, before doing that, a few informal remarks are in order. Structural types of the form $\Sigma := \coprod_{X_1,\ldots,X_n} F$ are meant to be interpreted as the sets of all tuples $(E_1, \ldots, E_n, f)$ where $E_1, \ldots, E_n$ are sets and f belongs to what would be the interpretation of F under the assumption that the type variables X_i are interpreted as the sets E_i. If we had a language of structural type theory with a type of integers $\mathbb{Z}$ and addition $+ : \mathbb{Z} \longrightarrow (\mathbb{Z} \longrightarrow \mathbb{Z})$ and we defined $\Sigma := \coprod_X X \longrightarrow (X \longrightarrow X)$, then $+@\Sigma : \Sigma$ would be interpreted as $(|\mathbb{Z}|, |+|)$ and it would belong to $|\Sigma|$, where, of course, we are using $|-|$ to represent the interpretation of a construction.

With these intentions in sight, there is no way that structural type theory could be directly interpreted in ZF set theory: structural types would be proper classes. However, interpreting it in NFU set theory seems more feasible, at least thanks to stratified comprehension. Thus, we will now introduce the weak semantics of any language of structural type theory by specifying how it can be interpreted, not in some arbitrary set theory, but in NFU in particular. We will do this by defining, for each construction c in the language of structural type theory, a stratified formula $\psi_c(\bar{c})$ characterising a non-empty set[7] $\bar{c}$. Given any construction c, we will informally refer to the set characterised by ψ_c as $|c|$. In this way, we shall write, informally, $|c| = |d|$ to mean $\forall \bar{c}, \bar{d}.\ (\psi_c(\bar{c}) \wedge \psi_d(\bar{d})) \rightarrow \bar{c} = \bar{d}$. Analogously, we will write $x \in |C|$ as an abbreviation for $\forall \bar{C}.\ \psi_C(\bar{C}) \rightarrow x \in C$.

To begin, we need to interpret all the atomic types in the language and the type of truth values. Each of these types C must be interpreted through a stratified formula $\psi_C(\bar{C})$ of the form $\forall x.\ x \in \bar{C} \leftrightarrow (\cdots)$ for some variable x; we require these formulas to have no free variables other than $\bar{C}$ and to characterise unique sets belonging to NSet^2. In particular, the type $*$ of truth values must be interpreted through a stratified formula $\psi_*(\bar{*})$ (of the aforementioned form) as a set with two distinct elements, 0 and 1. These two elements—which must be non-empty sets themselves—also need to be

[7]For reasons of technical convenience, we will define our interpretation in such a way that all constructions will be interpreted as sets, not as urelements. Consequently, no constructions will be interpreted as tuples, but some may be interpreted as singletons of tuples.

uniquely characterised by some stratified formulas $\psi_0(\bar{0})$ and $\psi_1(\bar{1})$ of the form that we have discussed. Given any type variable X, we set $\psi_X(\bar{X})$ to be the formula $\bar{X} = \tilde{X}$ for the free variable $\tilde{X}$; this formula is clearly stratified.

As we define our interpretation, we will assign to each type an *absolute height* $h(-)$. We will extend this function to all constructions by fixing, for every $a : A$, $h(a) := h(A) - 1$. In this way, we will show inductively that any formula $\psi_{(-)}$ characterising the interpretation of an object can be stratified with a relative height function σ such that, for any construction x with an interpretation $|x|$ represented by a variable $\bar{x}$ in $\psi_{(-)}$, we have $\sigma(\bar{x}) = h(x)$. Thus, to begin, we will assign an absolute height of 1 to atomic types, type variables and $*$, and, consequently, we will take the two truth values in $*$ to have an absolute height of 0.

Given any two types A and B for which we have already defined an interpretation, we will interpret the type $A \longrightarrow B$ as the set of functions from $\mathcal{P}_m|A|$ to $\mathcal{P}_n|B|$, where,[8] $m := h(B) \dot{-} h(A)$ and $n := h(A) \dot{-} h(B)$. We will define the absolute height of this type to be $h(A \longrightarrow B) := \max\{h(A), h(B)\} + 1$. The corresponding formula $\psi_{A \longrightarrow B}$ can be easily derived and shown to be stratified with a relative height function that agrees with h.

At this stage of the process, we can interpret all the constants of the language as elements of the interpretation of their corresponding types. This must be done through formulas $\psi_{(-)}$ characterising them uniquely in full analogy to the way in which we have interpreted types so far. Moreover, given any type A and any variable symbol x, we will set $\psi_{x_A}(\bar{x}_A)$ to be $\bar{x}_A = \tilde{x}_A$ for the free variable $\tilde{x}_A$; this formula is clearly stratified.

Given any two expressions a and b of the same type, we interpret $u := (a = b)$ as the set $\bar{u}$ characterised by

$$\begin{aligned}\psi_{a=b}(\bar{u}) := \forall x.\ x \in \bar{u} \leftrightarrow \forall \bar{a}, \bar{b}.\ \psi_a(\bar{a}) \wedge \psi_b(\bar{b}) \\ \rightarrow \big(\bar{a} = \bar{b} \rightarrow x \in |1|\big) \wedge \big(\bar{a} \neq \bar{b} \rightarrow x \in |0|\big)\end{aligned}$$

Incidentally, as previously mentioned, the formula $x \in |1|$ should be expanded as $\forall \bar{1}.\ \psi_1(\bar{1}) \rightarrow x \in \bar{1}$, and analogously for $x \in |0|$. The formula $\psi_{a=b}(\bar{u})$ is stratified provided that so are ψ_a, ψ_b, ψ_0 and ψ_1, for we can define a relative height function σ with $\sigma(\bar{u}) := 0 = h(a = b)$, $\sigma(\bar{a}) := h(a)$ and $\sigma(\bar{b}) := h(b)$.

Before moving on, we will introduce a small notational artefact. Let us consider any formula $\psi_c(\bar{c})$ characterising a certain construction c. Given

[8] Here we are using the *truncated subtraction operator* which takes any pair of natural numbers p, q to $p \dot{-} q := \max\{p - q, 0\}$.

some NFU variables $x_1, \ldots, x_n$ possibly appearing in ψ, and some terms $t_1, \ldots, t_n$ in the language of NFU, we will write $\psi_c(\bar{c}; x_1 = t_1, \ldots, x_n = t_n)$ in order to denote the formula

$$\forall x_1, \ldots, x_n \mathbf{.}\ (x_1 = t_1 \wedge \cdots \wedge x_n = t_n) \to \psi_c(\bar{c}).$$

Let A be a type and $b : B$ be any expression with possible instances of a free variable x_A. Fixing $m := h(B) \dot{-} h(A)$ and $n := h(A) \dot{-} h(B)$, the interpretation of $u := \lambda x_A \mathbf{.}\ b$ is defined by

$$\begin{aligned}\psi_{\lambda x_A .\ b}(\bar{u}) := \forall z \mathbf{.}\ z \in \bar{u} \leftrightarrow \exists x, y \mathbf{.}\ z = (\iota^m(x), \iota^n(y)) \\ \wedge\ x \in |A| \wedge \psi_b(y; \tilde{x}_A = x),\end{aligned}$$

Let us briefly explain why the defining formula of this set needs to be stratified. If we try to define a relative height function σ and we assume all the ψ's to be stratified, we only need to be concerned with having $\sigma(x) + m = \sigma(y) + n$. Nevertheless, according to our inductive hypothesis that relative height functions can always match h, we may assume that $\sigma(x) = \sigma(\tilde{x}_A) = h(A) - 1$ and $\sigma(y) = h(B) - 1$. Consequently,

$$\sigma(x) + m = h(A) + m - 1 = h(B) + n - 1 = \sigma(y) + n.$$

Furthermore, $\sigma(\bar{u}) = \sigma(x) + m + 1 = h(\lambda x_A \mathbf{.}\ b)$.

Let $f : A \longrightarrow B$ and $a : A$ be any two expressions for which we have defined ψ_f and ψ_a. For $u := f(a)$, we set

$$\begin{aligned}\psi_{f(a)}(\bar{u}) := \forall x \mathbf{.}\ x \in \bar{u} \leftrightarrow \forall \bar{a}, \bar{f} \mathbf{.}\ (\psi_a(\bar{a}) \wedge \psi_f(\bar{f})) \to \\ \forall y \mathbf{.}\ (\iota^m(\bar{a}), \iota^n(\bar{y})) \in \bar{f} \to x \in y\end{aligned}$$

Our inductive hypothesis readily implies that $\psi_{f(a)}$ needs to be stratified with a relative height function agreeing with h.

Let A be a type, $x_A : A$ be a variable and $\alpha : *$. We define the interpretation of $u := \epsilon x_A \mathbf{.}\ \alpha$ in terms of a fixed free variable Φ, common to all the formulas $\psi_{(-)}$, as

$$\begin{aligned}\psi_{\epsilon x_A .\ \alpha}(\bar{u}) := \forall z \mathbf{.}\ z \in \bar{u} \leftrightarrow \forall C \mathbf{.}\ \\ (\forall c \mathbf{.}\ c \in C \leftrightarrow (c \in |A| \wedge \forall \bar{\alpha} \mathbf{.}\ \psi_\alpha(\bar{\alpha}; \tilde{x}_A = c) \to \bar{\alpha} = |1|)) \to \\ \big(\exists \varphi \in \Phi \mathbf{.}\ \exists y \mathbf{.}\ \varphi = (C, y) \wedge \exists y_0 \mathbf{.}\ y_0 \in y \wedge z \in y_0\big) \\ \vee\ \big(\forall \varphi \in \Phi \mathbf{.}\ \forall y \mathbf{.}\ \varphi \neq (C, y) \wedge z \in \tilde{\perp}_A\big).\end{aligned}$$

We will soon see how Φ is meant to represent a global choice function NSet $\longrightarrow V$ taking any non-empty set x to an element of $\mathcal{P}_1(x)$.

Given a structural type $\Sigma := \coprod_{X_1,\ldots,X_n} F$, we define, for $m := h(F) - 1$,

$$\begin{aligned}\psi_\Sigma(\bar{\Sigma}) := \forall s.\ s \in \bar{\Sigma} \leftrightarrow \exists \hat{X}_1, \ldots, \hat{X}_n, \bar{F}, f.\ \hat{X}_1, \ldots, \hat{X}_n \in \mathrm{NSet}^2 \\ \wedge \psi_F(\bar{F}; \tilde{X}_1 = \hat{X}_1, \ldots, \tilde{X}_n = \hat{X}_n) \wedge f \in \bar{F} \\ \wedge s = \iota^1\left(\iota^m(\hat{X}_1), \ldots, \iota^m(\hat{X}_n), \iota^1(f)\right)\end{aligned}$$

We set $h(\Sigma) := h(F) + 2$ in order for ψ_Σ to be stratified with a relative height function agreeing with h. Of course, given any $u := s@\Sigma : \Sigma$ where $s : F[X_1 \| \chi_1, \ldots, X_n \| \chi_n]$ for some types χ_i, we set

$$\begin{aligned}\psi_{s@\Sigma}(\bar{u}) := \forall z.\ z \in \bar{u} \leftrightarrow \forall \bar{\chi}_1, \ldots, \bar{\chi}_n, \bar{s}.\ \psi_{\chi_1}(\bar{\chi}_1) \wedge \cdots \wedge \psi_{\chi_n}(\bar{\chi}_n) \wedge \psi_s(\bar{s}) \\ \rightarrow z = (\iota^m(\bar{\chi}_1), \ldots, \iota^m(\bar{\chi}_n), \iota^1(\bar{s})).\end{aligned}$$

This formula can be stratified with a relative height function σ such that $\sigma(\bar{u}) = h(s@\Sigma) = h(\Sigma) - 1$, but we cannot guarantee that σ will agree with h on any other free variables in $\psi_{s@\Sigma}$. Nevertheless, this is not an issue for, thanks to the restrictions in Rules 2 and 3d of Definition 3, we know that those free variables will not become bound in any other formula $\psi_{(-)}$ having $\psi_{s@\Sigma}$ as a subformula [11, § 8.3].

Lastly, given any such structural type Σ, a variable $x_F : F$ and an expression b of some type B, if $u := \lambda x_F @\Sigma.\ b$ can be constructed, then we set, for $m := h(B) \dot{-} h(\Sigma)$ and $n := h(\Sigma) \dot{-} h(B)$,

$$\begin{aligned}\psi_{\lambda x_F @\Sigma.\ b}(\bar{u}) := \forall z.\ z \in \bar{u} \leftrightarrow \exists x, y.\ z = (\iota^m(x), \iota^n(y)) \\ \wedge x \in |\Sigma| \wedge \exists \chi_1, \ldots, \chi_n, f. \\ x = \iota^1\left(\iota^{h(F)-1}(\chi_1), \ldots, \iota^{h(F)-1}(\chi_n), \iota^1(f)\right) \\ \wedge \psi_b(y; \tilde{x}_F = f, \tilde{X}_1 = \chi_1, \ldots, \tilde{X}_n = \chi_n).\end{aligned}$$

We have now defined a stratified formula ψ_c for every construction c in a language of structural type theory. By construction, given any type C with free instances of the type variables $X_1, \ldots, X_m$, the formula ψ_C will only have the free variables $\bar{C}$ and $\tilde{X}_1, \ldots, \tilde{X}_m$. Given any expression c, the formula ψ_c will include instances of the free variable Φ if c has instances of the ϵ operator, and, if c has free instances of some variables $z_1, \ldots, z_k$ and some type variables $X_1, \ldots, X_m$, then ψ_c will have free instances of the variables $\tilde{z}_1, \ldots, \tilde{z}_k$ and $\tilde{X}_1, \ldots, \tilde{X}_m$. In addition, for any subexpression of c of the form $(\epsilon x_A.\ \ldots)$, there can be free instances of $\tilde{\perp}_A$ in ψ_c. Any other variables in ψ_c will be bound except for $\bar{c}$.

In order to easily account for all the free variables that the choice operator may introduce into the formulas $\psi_{(-)}$, we will assume, for the remainder of

this section, that any expression with a subexpression of the form $(\epsilon x_A\mathbf{.}\ \ldots)$ has $\bot_A : A$ as a free variable.

The following result condenses how these formulas $\psi_{(-)}$ relate the constructions in structural type theory to the sets in NFU.

PROPOSITION 6. *Let us consider a language of structural type theory.*

(i) For any type C with free type variables $X_1, \ldots, X_m$, the formula

$$\forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2\mathbf{.}\ \exists \bar{C}\mathbf{.}\ S(\bar{C}) \wedge \psi_C(\bar{C})$$

is a theorem of NFU.

(ii) If $c : C$ is an expression with some free variables $z_0 : Z_0, \ldots, z_k : Z_k$ and free type variables $X_1, \ldots, X_m$, then[9]

$$\exists \Phi \in V^{\mathrm{NSet}}\mathbf{.}\ \big(\forall x \in \mathrm{NSet}\mathbf{.}\ \Phi(x) \in \mathcal{P}_1(x)\big) \wedge \forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2\mathbf{.}$$
$$\forall \tilde{z}_1, \bar{Z}_1 \ldots, \tilde{z}_k, \bar{Z}_k\mathbf{.}\ \big(\psi_{Z_1}(\bar{Z}_1) \wedge (\tilde{z}_1 \in \bar{Z}_1) \wedge \cdots \wedge \psi_{Z_k}(\bar{Z}_k) \wedge (\tilde{z}_k \in \bar{Z}_k)\big) \rightarrow$$
$$\exists \bar{c}\mathbf{.}\ S(\bar{c}) \wedge \psi_c(\bar{c}, \Phi)$$

is a theorem of NFU. This formula can be further rewritten, informally, as

$$\exists \Phi \in V^{\mathrm{NSet}}\mathbf{.}\ \big(\forall x \in \mathrm{NSet}\mathbf{.}\ \Phi(x) \in \mathcal{P}_1(x)\big) \wedge$$
$$\forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2\mathbf{.}\ \forall \tilde{z}_1 \in |Z_1|, \ldots, \tilde{z}_k \in |Z_k|\mathbf{.}\ \exists \bar{c}\mathbf{.}\ S(\bar{c}) \wedge \psi_c(\bar{c}, \Phi).$$

PROOF. Statement (i) follows directly from the axiom schema of stratified comprehension and the fact that the formulas ψ_C are stratified.

Regarding Statement (ii), we just have to take into account the axiom of choice, the axiom of stratified comprehension and the fact that, if some formulas of the form $\exists \Phi\mathbf{.}\ B$ and $A(\Phi)$ are theorems in a first-order theory, then so must be $\exists \Phi\mathbf{.}\ B \wedge A(\Phi)$ since

$$\vdash \{\exists \Phi\mathbf{.}\ B,\ A(\Phi)\} \vdash \{\exists \Phi\mathbf{.}\ B,\ \forall \Phi\mathbf{.}\ A(\Phi)\} \vdash \exists \Phi\mathbf{.}\ B \wedge A(\Phi).$$

■

DEFINITION 7. Let $\varphi : *$ be a formula of a language of structural type theory with some free variables $z_1 : Z_1, \ldots, z_k : Z_k$ and free type variables $X_1, \ldots, X_m$. Under a certain interpretation, φ is true if and only if

$$\forall \Phi \in V^{\mathrm{NSet}}\mathbf{.}\ \big(\forall x \in \mathrm{NSet}\mathbf{.}\ \Phi(x) \in \mathcal{P}_1(x)\big) \rightarrow \forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2\mathbf{.}$$
$$\forall \tilde{z}_1, \bar{Z}_1 \ldots, \tilde{z}_k, \bar{Z}_k\mathbf{.}\ \big(\psi_{Z_1}(\bar{Z}_1) \wedge (\tilde{z}_1 \in \bar{Z}_1) \wedge \cdots \wedge \psi_{Z_k}(\bar{Z}_k) \wedge (\tilde{z}_k \in \bar{Z}_k)\big) \rightarrow$$

[9] Given any two sets X and Y, we write Y^X to denote the set of functions from X to Y.

$$\forall \bar{\varphi}, \bar{1}.\ \psi_{\varphi}(\bar{\varphi}) \wedge \psi_1(\bar{1}) \rightarrow \bar{\varphi} = \bar{1},$$

which can be rewritten, informally as

$$\forall \Phi \in V^{\mathrm{NSet}}.\ \big(\forall x \in \mathrm{NSet}.\ \Phi(x) \in \mathcal{P}_1(x)\big) \rightarrow \forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2.$$
$$\forall \tilde{z}_1 \in |Z_1| \ldots, \tilde{z}_k \in |Z_k|.\ |\varphi| = |1|.$$

This amounts to stating that a formula is true if, for any choice function Φ, and for any values of the free type variables X_j in NSet^2 and of the free variables z_i in their respective domains $|Z_i|$, we have $|\varphi| = 1$.

It is straightforward to verify that, given any expression $c : C$ in a language of structural type theory under some interpretation, we must have $|c| \in |C|$ in the sense that, if $z_i : Z_i$ $(i = 1, \ldots, k)$ are the free variables in c and X_j $(j = 1, \ldots, m)$ are the free type variables appearing in c,

$$\forall \Phi \in V^{\mathrm{NSet}}.\ \big(\forall x \in \mathrm{NSet}.\ \Phi(x) \in \mathcal{P}_1(x)\big) \rightarrow \forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2.$$
$$\forall \tilde{z}_1 \in |Z_1| \ldots, \tilde{z}_k \in |Z_k|.\ \forall \bar{c}, \bar{C}.\ \psi_{\varphi}(\bar{c}) \wedge \psi_C(\bar{C}) \rightarrow \bar{c} \in \bar{C}$$

is a theorem of NFU.

Proposition 8. *Weak structural type theory is sound. Therefore, it is consistent relative to NFU set theory.*

Proof. (Proof sketch) The soundness proof of weak structural type theory is fully analogous in spirit to the soundness proof of simple type theory shown in a book by Andrews [3, § 54]. Nevertheless, one has to account for some of the technicalities introduced by our formal interpretation in NFU. Most notably, the book interprets simple type theory in an informal set theory (very much like we did back in Section 2.1), so the proof has to be translated into our formal NFU setting. In particular, the notion of a variable assignment function[10] is not explicit in our formal semantics in NFU, but it is implicit: in Definition 7, we can see how we rely on a bounded universal quantification over the variables $\tilde{z}_i$ and $\tilde{X}_j$ as a means of formally capturing the idea that, for a formula to be true, its interpretation has to be true for any variable assignment function (notice how this function would also give meaning to the type variables).

We will illustrate how this "translation into our NFU setting" is carried out by sketching the proof of how validity is preserved under the rule of substitution under equality.

[10] Variable assignment functions are introduced in Section 2.1. In Andrews' book, they are called, simply, "assignments".

As Andrews argues [3, Thm. 5402], in order to prove the preservation of validity of substitution under equality, it suffices to show that, under the assumption that $a = b$ is true (for some expressions a and b), then $\varphi = \varphi'$ must be true for any expression φ and for any expression φ' resulting from replacing at most one instance of a in φ by b, provided that such an instance was not a variable immediately preceded by λ or ϵ.

It follows from Definition 7 that—given any two expressions α and β of the same type with free variables $z_1 : Z_1, \ldots, Z_k$ and free type variables $X_1, \ldots, X_m$—the formula $\alpha = \beta$ is true if and only if

$$\forall \Phi \in \mathrm{NSet} \longrightarrow V\mathbf{.}\ \big(\forall x \in \mathrm{NSet}\mathbf{.}\ \Phi(x) \in \mathcal{P}_1(x)\big) \to \forall \tilde{X}_1, \ldots, \tilde{X}_m \in \mathrm{NSet}^2\mathbf{.}$$
$$\forall \tilde{z}_1 \in |Z_1| \ldots, \tilde{z}_k \in |Z_k|\mathbf{.}\ \forall x\mathbf{.}\ \psi_\alpha(x) \leftrightarrow \psi_\beta(x).$$

is a theorem of NFU. For the sake of brevity, we will abbreviate this as $Q\mathbf{.}\ \psi_\alpha(x) \leftrightarrow \psi_\beta(x)$. Notice that Q is representing the application of a sequence of possibly bounded universal quantifiers.

Since $a = b$ is assumed to be true, the hypothesis that we should work with is $Q\mathbf{.}\ \psi_a \leftrightarrow \psi_b$ and, from there, we need to prove that $Q\mathbf{.}\ \psi_\varphi \leftrightarrow \psi_{\varphi'}$. As in the book, proceed by induction on the construction of φ considering two base cases: first, that φ is identical to φ'; second, that φ is a and φ' is b. These cases are trivial. For the inductive steps, one only has to work—using some elementary first-order logic—with the formulas $\psi_{(-)}$ characterising the interpretation of all the constructions in our language. This is how substitution under equality can be shown to preserve validity in our formal framework.

The only missing ingredient for soundness is the validity of the axioms of weak structural type theory. Its verification is mechanical yet lengthy, so we will not discuss it. We should nevertheless mention that the verification is greatly simplified by dropping some universal quantifiers from the axiom schemata using Results 5215 (universal instantiation) and 5220 (rule of universal generalisation) in Andrews' book [3, § 52]. ■

4. Identification Under Isomorphism

In the previous section, we introduced the language of structural type theory together with its weak semantics and a sound deductive system with respect to those semantics. In this section, we will introduce the "strong" semantics of structural type theory, under which the equality of two structures (within structural types) will be interpreted as their being isomorphic. Moreover, we will introduce a stronger formal system that will enable us to exploit

this, and we will prove its soundness with respect to the strong semantics of structural type theory.

But, if we wish to proceed in this fashion, the first thing that we need to do is to define, precisely, what it means for two structures in the language of structural type theory to be isomorphic.

Definition 9. Let $X_1, \ldots, X_n$ be type variables and let $A_1, B_1, \ldots, A_n, B_n$ be types. For any type M, we will denote $M^A := M[X_1 \| A_1, \ldots, X_n \| A_n]$ and $M^B := M[X_1 \| B_1, \ldots, X_n \| B_n]$.

Given a collection of n bijections $\psi_i : A_i \longrightarrow B_i$ and any type M with no instances of $\coprod$, we shall define a *transport function*

$$\tau^{\psi_1,\ldots,\psi_n}_{X_1,\ldots,X_n;M} : M^A \longrightarrow M^B,$$

which we will denote as τ_M when doing so leads to no ambiguity. We will define this transport function by induction on the construction of M.

As base case, if M is one of the type variables X_i, we set $\tau_M := \psi_i$. Otherwise, if M is a type with no instances of any of the type variables X_i, we set τ_M to be the identity function in M (i.e., $\lambda x_M .\ x_M$).

As inductive step, if M is of the form $U \longrightarrow V$ for two types U and V for which a transport function has been defined, we set, for a variable symbol f,

$$\tau_{U \longrightarrow V} := \lambda f_{U^A \longrightarrow V^A} .\ \left(\tau_V \circ f \circ \tau_U^{-1}\right).$$

Let $\Sigma := \coprod_{X_1,\ldots,X_n} F$ be a structural type. Given some $f^A : F^A$ and $f^B : F^B$, we will say that $f^A @ \Sigma$ and $f^B @ \Sigma$ are *isomorphic* if there exist n bijections $\psi_i : A_i \longrightarrow B_i$ such that

$$\tau^{\psi_1,\ldots,\psi_n}_{X_1,\ldots,X_n;F}(f^A) = f^B.$$

To put this in more formal terms, for any such Σ, f^A and f^B, let $\psi_i : A_i \longrightarrow B_i$ be variables (for $i = 1, \ldots, n$). We construct the formula

$$\text{isomorphic}[f^A @ \Sigma, f^B @ \Sigma] := \exists \psi_1 .\ \ldots \exists \psi_n .\ \text{bijective}[\psi_1] \wedge \cdots \wedge \text{bijective}[\psi_n] \wedge \tau^{\psi_1,\ldots,\psi_n}_{X_1,\ldots,X_n;F}(f^A) = f^B,$$

where bijective$[-]$ should be unwrapped as per its definition in Section 2.1. The construction isomorphic$[f^A, f^B]$ represents a formula in the language of structural type theory. Two structures $f^A @ \Sigma$ and $f^B @ \Sigma$ are defined to be isomorphic if isomorphic$[f^A @ \Sigma, f^B @ \Sigma]$ holds.

The formal system of *(strong) structural type theory* is the formal system of weak structural type theory with this additional axiom:

(T7) Given any structural type Σ and any $s@\Sigma, t@\Sigma : \Sigma$, if $s@\Sigma$ and $t@\Sigma$ are isomorphic, then $s@\Sigma = t@\Sigma$. Formally,

$$\text{isomorphic}[s@\Sigma, t@\Sigma] \rightarrow s@\Sigma = t@\Sigma.$$

REMARK 10. We have defined transport functions as objects of simple type theory, but the same notion can be defined for set-theoretical functions.

Let us say that we have some pairs of sets $A_1, B_1, \ldots, A_n, B_n$ and some bijections $\theta_i : A_i \longrightarrow B_i$, and let us consider a structural type $\Sigma := \coprod_{X_1,\ldots,X_n} F$. In order to define a set-theoretical transport function, we could incorporate into our language some atomic types $A_1^0, B_1^0, \ldots, A_n^0, B_n^0$ and some atomic constants $\theta_i^0 : A_i^0 \longrightarrow B_i^0$. The function $\tau^{\theta_1,\ldots,\theta_n}_{X_1,\ldots,X_n;F}$ would be defined as the interpretation of

$$\tau^{\theta_1^0,\ldots,\theta_n^0}_{X_1,\ldots,X_n;F}$$

under $|A_i^0| = A_i$, $|B_i^0| = B_i$ and $|\theta_i^0| = \theta_i$.

In the interest of clarity and brevity, we present the following fundamental result using the weak semantics of simple type theory informally. The reader should have no difficulty—beyond the unavoidable tediousness of the process—in formalising this result and its proof in the language of NFU set theory using the formal definition of the weak semantics that we introduced previously.

THEOREM 11. *(Transport action) Let $X_1, \ldots, X_n$ be type variables and let $A_1, B_1, \ldots, A_n, B_n$ be types. Let us consider n bijections $\psi_i : A_i \longrightarrow B_i$. In what follows, we will assume that all transport functions are defined with respect to these bijections and to the type variables $X_1, \ldots, X_n$.*

Given any construction C with free variables $z_1 : Z_1, \ldots, z_k : Z_k$, we will denote

$$C^A := C[X_1 \| A_1, \ldots, X_n \| A_n]_{\mathsf{f}},$$
$$C^B := C[z_1 \| \tau_{Z_1}(z_1^A), \ldots, z_k \| \tau_{Z_k}(z_k^A), X_1 \| B_1, \ldots, X_n \| B_n]_{\mathsf{f}}.$$

Under these hypotheses, for any expression $\alpha : M$ with no instances of the choice operator and no occurrences of @, and such that M has no instances of $\coprod$, we have

$$\left|\tau_M(\alpha^A)\right| = \left|\alpha^B\right|.$$

where we use $|-|$ to refer to the (weak) semantic interpretation of a construction.

Proof. We should firstly remark that the expressions α^A and α^B are always well-defined under our current hypotheses. It is also worth pointing out that, since M is assumed not to have any instances of $\coprod$, the transport function τ_M is well-defined by Definition 9. We now proceed with a proof by induction on the construction of α.

If α is a variable of any type, then the result is obvious. If α is an expression with no free variables and no free instances of the types variables X_i, then α^A will be identical to α^B; furthermore, M will have no instances of the type variables X_i, so τ_M will behave like the identity function. Thus,

$$|\tau_M(\alpha^A)| = |\alpha^A| = |\alpha^B|,$$

and the result holds.

If α is $\beta(t)$ for some $\beta : T \longrightarrow M$ and $t : T$ for which the inductive hypothesis is true, then

$$\begin{aligned}\left|\beta^B(t^B)\right| &= \left|\left(\tau_{M\longrightarrow T}(\beta^A)\right)\left(\tau_T(t^A)\right)\right| = |\tau_M \circ \beta^A \circ \tau_T^{-1} \circ \tau_T(t^A)| \\ &= |\tau_M \circ \beta^A(t^A)|,\end{aligned}$$

as we wanted to show. Regarding equality, if M is $*$ and α is of the form $s = t$ for some $s, t : T$ satisfying the inductive hypothesis, we have

$$\left|\tau_*(s^A = t^A)\right| = |s^A = t^A| = |\tau_T(s^A) = \tau_T(t^A)| = |s^B = t^B|,$$

where we have used the fact that all transports functions are bijective and that τ_* is the identity function, because $*$ has no instances of any of the type variables X_i.

Lastly, we will prove the result for an expression α of the form $(\lambda x_S.\ \beta) : S \longrightarrow T$ with $\beta : T$ verifying the result. If the free variables of β are $x_S : S, z_1 : Z_1, \ldots, z_k : Z_k$, then

$$\left|\tau_{S\longrightarrow T}\left((\lambda x_S.\ \beta)^A\right)\right| = \left|\tau_T \circ (\lambda x_S.\ \beta)^A \circ \tau_S^{-1}\right| = \left|(\lambda x_{S^A}.\ \tau_T(\beta^A)) \circ \tau_S^{-1}\right|.$$

Using the inductive hypothesis on β, this must be equal to

$$\left|(\lambda x_{S^A}.\ \beta[x_S \| \tau_S(x_{S^A}), z_1 \| \tau_{Z_1}(z_1^A), \ldots, z_k \| \tau_{Z_k}(z_k^A), X_1 \| B_1, \ldots, X_n \| B_n]_{\mathfrak{f}}) \circ \tau_S^{-1}\right|,$$

but, using the semantics of our language, this can be simplified to

$$\left|(\lambda x_{S^B}.\ \beta[x_S \| x_{S^B}, z_1 \| \tau_{Z_1}(z_1^A), \ldots, z_k \| \tau_{Z_k}(z_k^A), X_1 \| B_1, \ldots, X_n \| B_n]_{\mathfrak{f}}) \circ \tau_S \circ \tau_S^{-1}\right|,$$

which is equal to $|(\lambda x_S.\ \beta)^B|$, as we wanted to show. ∎

Scholium 12. *In the transport action theorem, if we allowed α to have instances of the choice operator ϵ, the result would no longer hold. As a counterexample, take a type A with more than two elements and consider a bijection $\psi : A \longrightarrow A$ with no fixed points. Set, for a type variable X,*

$\alpha := (\epsilon x_X .\ x_X = x_X)$ *and*

$$\alpha^A := \alpha^B := (\epsilon x_A .\ x_A = x_A).$$

These two expressions would satisfy the hypothesis of the theorem (except for the choice operator restriction) with $n := 1$, $X_1 := X$, $A_1 := A$, $B_1 := A$ *and* $\psi_1 := \psi$. *However,*

$$\left|\tau^{\psi}_{X;X}(\alpha^A)\right| = \left|\psi_1(\alpha^A)\right| \neq \left|\alpha^B\right|$$

for $\alpha^A = \alpha^B$ *and* ψ *is assumed not to have any fixed points.*

This is why Condition 3a was introduced in Definition 3.

COROLLARY 13. *(Transport invariance) Let* $\Sigma := \coprod_{X_1,\ldots,X_n} F$ *be a structural type and let*

$$\eta := (\lambda x_F @\Sigma .\ \beta) : \Sigma \longrightarrow M$$

be an expression in the language of structural type theory. Given some types $A_1, B_1, \ldots, A_n, B_n$, *let* $a : F[X_1 \| A_1, \ldots, X_n \| A_n]$ *and* $b : F[X_1 \| B_1, \ldots, X_n \| B_n]$ *be a pair of expressions. Clearly,* $a@\Sigma, b@\Sigma : \Sigma$.

If $a@\Sigma$ *and* $b@\Sigma$ *are isomorphic (i.e., if* isomorphic$[a@\Sigma, b@\Sigma]$ *holds), then*

$$|\eta(a@\Sigma)| = |\eta(b@\Sigma)|\,.$$

PROOF. Since $a@\Sigma$ and $b@\Sigma$ are isomorphic, there must exist some bijections $\psi_i : A_i \longrightarrow B_i$ (for $i = 1, \ldots, n$) such that

$$\tau^{\psi_1,\ldots,\psi_n}_{X_1,\ldots,X_n;F}(a) = b.$$

For the remainder of the proof, all our transport functions will be defined with respect to these bijections (ψ_i) and these type variables (X_i).

According to conditions 3a–3d in Definition 3, the expression β can have no instances of ϵ and @. Therefore, thanks to the transport action theorem (Theorem 11), we know that

$$\left|\tau_M\big(\beta[x_F \| a, X_1 \| A_1, \ldots, X_n \| A_n]_{\mathsf{f}}\big)\right| = \left|\beta[x_F \| \underbrace{b}_{\tau_F(a)}, X_1 \| B_1, \ldots, X_n \| B_n]_{\mathsf{f}}\right|,$$

which, using (T6), implies that

$$\left|\tau_M(\eta(a@\Sigma))\right| = \left|\eta\,(b@\Sigma)\right|.$$

However, by condition 3b in definition 3, we know that M must have no instances of the type variables X_i, hence τ_M must be the identity function.

It then follows that

$$|\eta(a@\Sigma)| = |\eta(b@\Sigma)|\,,$$

and thus the proof is complete. ■

This last result shows that we may perfectly assume isomorphic structures to be equal within structural types, all without putting at risk the well-definedness of the constructions of our language: in particular, that of functions having structural types as their domain. In order to take advantage of this possibility, we will redefine the semantics of our language in order to reinterpret structural types—not as the set of all the structures of a certain type, but as the quotient of such a set under the equivalence relation of being isomorphic. Let us make this idea precise.

We are about to introduce the *strong semantics* of the language of structural type theory. In what follows, we will use $|-|_w$ to denote the weak interpretation of a construction (i.e., its interpretation in the weak semantics) and $|-|$ to denote its strong interpretation.

Let $\Sigma := \coprod_{X_1,\ldots,X_n} F$. The strong interpretation of Σ is the quotient set on $|\Sigma|_w$ under the equivalence relation that relates any pair of elements $(x_1,\ldots,x_n,f),(y_1,\ldots,y_n,g) \in |\Sigma|_w$ if and only if there exist some bijections[11] $\theta_i : \pi^{h(F)-1}(x_i) \longrightarrow \pi^{h(F)-1}(y_i)$ for $i = 1,\ldots,n$ such that

$$\tau^{\theta_1,\ldots,\theta_n}_{X_1,\ldots,X_n;F}(\pi^1(f)) = \pi^1(g)$$

(cf. Remark 10). It is straightforward to check that this defines an equivalence relation on $|\Sigma|_w$, hence we know that its quotient set must exist. As we mentioned previously, we take such a quotient set to be $|\Sigma|$.

Reinterpreting structural types Σ also forces us to reinterpret all the expressions of the form $f@\Sigma$ and $\lambda x@\Sigma.\ b$. The former is easy to reinterpret, for we only need to take $|f@\Sigma|$ to be the equivalence class of $|f@\Sigma|_w$. Moreover—letting B be the type of b—$|\lambda x@\Sigma.\ b|$ can just be taken to be the function from $|\Sigma|$ to $|B|$ mapping any equivalence class in $|\Sigma|$ to its image under $|x@\Sigma.\ b|_w$. Indeed, this will be a well-defined function thanks to transport invariance (Corollary 13).

The remaining constructions of the language are interpreted in the strong semantics in the same way as in the weak semantics.

The Axiom Schemata (T1–T7) are valid in the strong semantics of structural type theory, so the relative consistency of the system with respect to

[11] Recall that, because of the way in which we defined the weak semantics, in the tuple $(x_1,\ldots,x_i,f) \in |\Sigma|_w$, f is the result of applying ι^1 on a set and each x_i is the result of applying $\iota^{h(F)-1}$ on a set.

NFU follows. In particular, the validity of (T7) is a direct consequence of how structural types are reinterpreted in the strong semantics, and of the fact that—apart from structural types and their associated constructions—the remaining constructions of the language are interpreted in the strong semantics in the same way as in the weak semantics.

5. Practical Considerations

5.1. Further Extensions: Cartesian Products

The theory that we have presented is intentionally minimal to the end of facilitating the discussion of its properties. However, simple type theory is a very versatile language that can be easily extended in order to enrich its expressivity; the reader may refer to Farmer's article [8] for an extensive survey of possible extensions. While extending the language would make the study of the system itself more cumbersome, it can prove to be useful—or even essential—in scenarios where one has to work within the formal system directly; for example, when using it in a proof assistant.

Finite Cartesian products can, in principle, be implemented solely with function types and with a type of indices. However, it can be handy to have them built into the language; there are already a few extensions of simple type theory that accomplish this [7]. If we incorporated finite Cartesian products into the language of structural type theory, they could be effortlessly interpreted in NFU. Moreover, we could also extend our definition of transport functions in order to be able to use Cartesian products in the construction of structural types while still being able to capture what it means for two structures in these types to be isomorphic. Suppose that $\iota_{A_1 \times A_2} : A_1 \longrightarrow (A_2 \longrightarrow A_1 \times A_2)$ is the type-theoretical inclusion function of a Cartesian product $A_1 \times A_2$ and that $\pi_i^{A_1 \times A_2} : A_1 \times A_2 \longrightarrow A_i$ are its two corresponding projections (the behaviour of these inclusion and projection functions could be defined axiomatically). If we had already defined the transport functions on some types M and N with respect to some type variables $X_1, \ldots, X_n$ and some bijections $\theta_i : A_i \longrightarrow B_i$, we could define the transport function on $M \times N$ as

$$\tau^{\theta_1,\ldots,\theta_n}_{X_1,\ldots,X_n;M\times N} := \lambda x_{M^A \times N^A} .\ \iota_{M^B \times N^B} \left(\tau_M(\pi_1^{M^A \times N^A}(x)), \tau_N(\pi_2^{M^A \times N^A}(x)) \right)$$

where, as usual, we are denoting $T^A := T[X_1 \| A_1, \ldots, X_n \| A_n]$ and $T^B := T[X_1 \| B_1, \ldots, X_n \| B_n]$ for any type T. It can be easily shown that, with this extension of transport functions, Theorem 11 would still hold while

allowing the type M in the theorem's hypothesis to include cartesian products.

In this way, Cartesian products of types could also be added to (strong) structural type theory and employed in the construction of structural types.

5.2. A Simple Example

To better illustrate how structural type theory could be used in practice, let us consider a simple example. For the purposes of this discussion, we will work with a language of structural type theory incorporating an atomic type $\mathbb{Z}$ and some atomic constants $0, 1 : \mathbb{Z}$ and $+ : \mathbb{Z} \longrightarrow (\mathbb{Z} \longrightarrow \mathbb{Z})$. Here, we are using currying in order to encode addition and multiplication, but we could as well consider an extension of structural type theory with Cartesian products as discussed in the preceding subsection. In addition to this, we will assume our formal system to have been extended with all the usual axioms of arithmetic for $\mathbb{Z}, 0, 1, +$. In particular, those axioms should include the associativity of integer addition, i.e.,

$$\forall x_{\mathbb{Z}}, y_{\mathbb{Z}}, z_{\mathbb{Z}}.\ x + (y + z) = (x + y) + z. \tag{assoc}$$

The type of all magmas is defined as

$$\mathrm{Magma} := \coprod_{X} X \longrightarrow (X \longrightarrow X),$$

and, clearly, $+@\mathrm{Magma} : \mathrm{Magma}$. On this type of magmas, we can define the predicate isSemigroup as an expression of type $\mathrm{Magma} \longrightarrow *$ given by

$$\mathrm{isSemigroup} := (\lambda f@\mathrm{Magma}.\ \forall x_X, y_X, z_X.\ f(f(x)(y))(z) = f(x)(f(y)(z))).$$

for some variable $f : X \longrightarrow (X \longrightarrow X)$ and some variable symbols x, y, z.

In our case, it can be easily deduced from the axioms that isSemigroup $(+@\mathrm{Magma})$ would be a theorem. More precisely, an instantiation of the axiom schema (T6) yields

$$\mathrm{isSemigroup}(+@\mathrm{Magma}) = \forall x_{\mathbb{Z}}, y_{\mathbb{Z}}, z_{\mathbb{Z}}.\ x + (y + z) = (x + y) + z.$$

So, using substitution under equality with (assoc), it follows that isSemigroup $(+@\mathrm{Magma})$ is a theorem.

On a different note, thanks to (T7), equality between any two structures in Magma, is always satisfied whenever those two structures are isomorphic. What is more, unwrapping our definition of isomorphism, it can be easily checked that two structures $f@\mathrm{Magma}$ and $g@\mathrm{Magma}$ (with $f : A \longrightarrow (A \longrightarrow A$ and $g : B \longrightarrow (B \longrightarrow B)$ for some types A and B) are isomorphic

if and only if there is a bijection $\theta : A \longrightarrow B$ such that

$$\forall x_A, y_A.\ \theta\big(f(x)(y)\big) = g\big(\theta(x)\big)\big(\theta(y)\big).$$

This, of course, agrees with the usual algebraic notion of isomorphism.

We know that any function defined from the type Magma to *any* other type will be invariant under isomorphism—we showed this in Corollary 13, and it is the very reason why structural type theory is consistent (relative to NFU set theory). This applies, in particular, to predicates. For example, the predicate isSemigroup is, unsurprisingly, invariant under isomorphism. Thus, if a magma is a semigroup, we can directly deduce that any isomorphic magma will also be one. This is all built into the theory.

5.3. Comparison to Bourbaki's Theory of Structures

A general, formal definition of the concept of mathematical structure and of some of its intrinsically related notions, such as that of isomorphism, can be found in Bourbaki's theory of structures, whose significance has been discussed in an article by Marquis [13]. In this subsection, we compare our theory to that of Bourbaki and, to that end, we invoke and mention some of its core ideas. A detailed exposition of them can be found in Bourbaki's work [4, Ch. IV].

Our structural types can be seen as analogues of a "species of structures" without axioms. In particular, if we are given a structural type of the form $\coprod_{X_1,\ldots,X_n} F$, the types $X_1, \ldots, X_n$ could be identified with the principal base sets, and the auxiliary base sets would correspond to all the instances of atomic types different from the type variables X_i within F. Lastly, the typification of the species would be the identified with the construction of the type F itself.

In addition to this, the way in which Bourbaki defines what it means for two structures to be isomorphic is perfectly analogous to ours. Given a species of structures with n principal base sets, and some bijections $f_1, \ldots, f_n$ between the principal base sets of two structures U and U', these bijections can be "canonically extended" with the "echelon construction scheme" of the typification of the species of structures. If the image of U under the resulting extension is U', then $(f_1, \ldots, f_n)$ is an isomorphism between U and U' and the structures are said to be isomorphic. In our case, given a structural type $\coprod_{X_1,\ldots,X_n} F$ and some bijections $\theta_1, \ldots, \theta_n$, this canonical extension can be identified with the transport function on F. Indeed, this transport function does no other thing than extending the bijections from the domains of a structure (principal base sets in Bourbaki's language) to

other types (through a echelon construction scheme, in Bourbaki's terms) in a manner analogous to the way in which canonical extensions work.

Of course, the approach taken by Bourbaki is fundamentally different from the one we have presented. Their theory of structures is built on top of set theory and, in that sense, is essentially metatheoretical. Ours, instead, is built into a formal system, relying on the expressive power of simple type theory, while still having set-theoretical semantics.

6. Conclusions and Future Work

In this paper, we have introduced structural type theory: an extension of type theory in which it is possible to work with mathematical structures within some types that we have referred to as "structural types", and to identify isomorphic structures in those types. In order to do this, we have introduced a general definition for the notion of isomorphism.

The possibility of identifying isomorphic structures in structural types could have given rise to issues relating to the well-definedness of functions from these structural types. Nevertheless, we addressed these potential problems with our transport invariance result (Corollary 13), which essentially showed that, thanks to the syntactical restrictions of the language of structural type theory, any function defined on structural types—and, in particular, any predicate on them—needs to be invariant under isomorphism.

In addition to this, throughout the paper, we have defined some set-theoretical semantics for structural type theory in order to be able to interpret its language within NFU set theory. We first introduced some weak semantics (in which isomorphic structures could not yet be identified) and, upon that foundation, we were able to formulate the transport invariance result which showed how the strong semantics of the language (in which isomorphic structures could be identified) were well-defined.

We have also covered a few practical matters, such as briefly touching upon how structural type theory could be extended with Cartesian products of types, exemplifying the use of structural type theory with some simple constructions, and drawing some analogies between structural type theory and Bourbaki's theory of structures.

In regard to future work, the author is currently working on the computer implementation of structural type theory. A matter that remains to be explored is whether some of the syntactical restrictions introduced in Definition 3—which we have shown to be sufficient for the well-definedness of structural type theory—can be further relaxed.

Acknowledgements. I am indebted to Prof. Elías F. Combarro and Prof. Ignacio F. Rúa for their mentorship and guidance. Were it not for them, this paper would have never been written. I would also like to thank the anonymous reviewers for their generous work. Their suggestions, remarks and corrections were extraordinarily exhaustive and helpful.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature.

Open Access. This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

References

[1] Adlešić, T., and V. Čačić, The cardinal squaring principle and an alternative axiomatization of NFU, *Bulletin of the Section of Logic* 52(4):551–581, 2023.

[2] Andrews, P.B., A reduction of the axioms for the theory of propositional types, *Fundamenta Mathematicae* 52:345–350, 1963.

[3] Andrews, P.B., *An Introduction to Mathematical Logic and Type Theory: to Truth Through Proof*, 2nd edn., Springer, Berlin, 2002.

[4] Bourbaki, N., *Theory of Sets*, Springer, Berlin, 2004.

[5] Church, A., A formulation of the simple theory of types, *The Journal of Symbolic Logic* 5(2):56–68, 1940.

[6] Farmer, W.M., A simple type theory with partial functions and subtypes, *Annals of Pure and Applied Logic* 64:211–240, 1993.

[7] Farmer, W.M., *A basic extended simple type theory*, SQRL Report No. 14, McMaster University, 2003.

[8] Farmer,W.M., The seven virtues of simple type theory, *Journal of Applied Logic* 6(3):267–286, 2008.

[9] Girard, J.Y., Y. Lafont, and P. Taylor, *Proofs and Types*, vol. 7 of *Cambridge Tracts in Theoretical Computer Science*, Cambridge University Press, Cambridge, 1989.

[10] Henkin, L., A theory of propositional types, *Fundamenta Mathematicae* 52:323–344, 1963.

[11] Holmes, R., *Elementary Set Theory with a Universal Set*, Cahiers du Centre de Logique, vol. 10, Academia-L'Harmattan, Louvain, 1998.

[12] Jensen, R.B., On the consistency of a slight (?) modification of Quine's "New Foundations". *Synthese* 19:250–263, 1969.

[13] Marquis, J.-P., Forms of Structuralism: Bourbaki and the Philosophers, in A. Peruzzi, and S.Z. Caiani, (eds.), Structures *Mères: Semantics, Mathematics, and Cognitive Science*, vol. 57 of *Studies in Applied Philosophy, Epistemology and Rational Ethics*, Springer Cham, 2020, pp. 37–57.

[14] Reynolds, J.C., Towards a theory of type structure, in B. Robinet, (ed.), *Programming Symposium*, Springer, Berlin, Heidelberg, 1974, pp. 408–425.

[15] The Univalent Foundations Project, *Homotopy Type Theory: Univalent Foundations of Mathematics*, Institute for Advanced Study, 2013.

S. González-Castillo
Department of Mathematics
University of Oviedo
Oviedo Asturias
Spain
samuelgc@uniovi.es